

Logos Experiment 7

Deeper-Narrower Transformer - Fixed-Budget Architecture Comparison

1. Experiment Purpose

This experiment tested whether a deeper but narrower decoder-only Transformer can improve parameter efficiency for the Logos language-modeling project. The original baseline used 8 layers with 256-dimensional embeddings. Experiment 7 changed the shape to 12 layers with 224-dimensional embeddings while keeping the same training objective, tokenizer, dataset size, context length, batch size, and training budget.

The main research question was: can extra depth compensate for reduced width and produce similar or better validation quality with fewer parameters?

Scientific status	Result
Experiment type	Controlled architecture-shape test
Main variable changed	Depth/width shape: 8 x 256 changed to 12 x 224
Control baseline	Original decoder-only GPT-style baseline
Conclusion	Reduced model size slightly, but worsened quality and efficiency under the fixed budget

2. Experimental Setup

Component	Experiment 7 setup
Run name	logos_exp07_deeper_narrower
Model family	Decoder-only causal Transformer
Dataset	OpenWebText subset
Tokenizer	GPT-2 tokenizer
Training objective	Next-token prediction
Architecture	Deeper-narrower Transformer
Layers	12
Embedding dimension	224
Attention heads	7
Head dimension	32
Context length	256 tokens
Batch size	64
Tokens per step	16,384
Feed-forward network	Standard ReLU FFN
Normalization	LayerNorm
Weight tying	False
Training iterations	1500

3. Final Results

Metric	Value
Parameters	29,875,409
Best iteration	1500
Best validation loss	5.8239
Final train loss	5.6724
Final validation loss	5.8030
Final validation perplexity	331.29
Checkpoint size	114.02 MB
Peak VRAM	11.11 GB
Training time	178.21 sec, about 2.97 min
Training speed	138,992 tokens/sec
Generation speed	73.27 tokens/sec
Generation latency	13.65 ms/token
Average GPU power	244.58 W
Estimated GPU energy	11.97 Wh

4. Training Behavior

The model continued improving until the final evaluation point, with the best validation loss occurring at iteration 1500. This means the model had not clearly saturated by the end of the run. However, because all experiments use the same 1500-iteration budget, this still provides a fair fixed-budget comparison.

Iteration	Train loss	Validation loss	Validation perplexity
0	10.8759	10.8752	52851.74
150	7.0464	7.0782	1185.89
300	6.4100	6.5041	667.90
600	6.0327	6.1522	469.75
900	5.7884	5.9460	382.21
1200	5.6921	5.8373	342.84
1350	5.6667	5.8336	341.59
1500	5.6579	5.8239	338.28

5. Comparison with Original Baseline

Metric	Original baseline	Experiment 7	Interpretation
Layers	8	12	Deeper
Embedding dimension	256	224	Narrower
Attention heads	8	7	Narrower attention stack
Parameters	32.16M	29.88M	7.10% fewer parameters
Final val loss	5.7634	5.8030	Worse quality
Final perplexity	318.42	331.29	4.04% higher perplexity
Checkpoint size	122.71 MB	114.02 MB	7.08% smaller
Peak VRAM	10.84 GB	11.11 GB	Slightly higher
Training time	156.42 sec	178.21 sec	13.93% slower
Training speed	about 158k tok/sec	138,992 tok/sec	Lower throughput
Generation latency	10.20 ms/token	13.65 ms/token	33.80% slower
Energy	10.78 Wh	11.97 Wh	11.05% higher

6. Scientific Interpretation

The deeper-narrower architecture reduced parameters and checkpoint size, but it did not improve the capability-resource tradeoff. The validation loss and perplexity both worsened compared to the original baseline. The model also trained more slowly, generated more slowly, used slightly more VRAM, and consumed more estimated energy.

This suggests that, under the current short fixed-budget setting, the extra depth did not compensate for the reduced embedding width. More layers introduce additional sequential computation, which can slow training and inference. The smaller width reduced the parameter count, but not enough to offset the quality and speed losses.

Observation	Meaning
Parameter count decreased	The shape change succeeded at reducing model size slightly.
Validation loss increased	The architecture was less effective for this short training budget.
Training throughput decreased	More layers increased sequential computation cost.
Generation latency increased	More Transformer blocks made autoregressive decoding slower.
Energy increased	Longer runtime outweighed the modest parameter reduction.
Checkpoint size decreased	Storage efficiency improved, but less strongly than weight tying.

7. Conclusion

Experiment 7 is not a successful optimization for the current Logos setup. It achieved a modest 7.10% reduction in parameter count and a 7.08% reduction in checkpoint size, but it worsened validation loss, perplexity, speed, latency, VRAM usage, and energy consumption. Therefore, the original 8-layer, 256-dimensional baseline remains better than this deeper-narrower variant under the same fixed training budget.

8. Best Model Status After This Experiment

Rank	Experiment	Reason
Best overall	Experiment 5: Weight Tying + SwiGLU	Lowest final validation loss and perplexity with about 40% fewer parameters than baseline
Best storage reduction	Experiment 1 or 5: Weight Tying variants	Reduced checkpoint size from about 122.71 MB to about 73.44 MB
Best inference latency	Experiment 4: SwiGLU FFN	Measured 7.78 ms/token generation latency
Not recommended	Experiment 7: Deeper-Narrower	Smaller checkpoint, but worse quality and slower runtime

9. Notes for Research Documentation

This experiment is still useful even though it did not improve performance. Negative results help identify which optimization directions are not promising under the current constraints. The result shows that simply increasing depth while reducing width is not enough to improve efficiency in this small, short-trained Logos setting.

Point	Documentation note
Hypothesis	A deeper-narrower Transformer might achieve similar or better quality with fewer parameters.
Outcome	The hypothesis was not supported by this run.
Main lesson	Depth can increase sequential cost and latency even when parameter count decreases.
Possible future retest	Try longer training, different learning rate, or combine depth changes with weight tying/SwiGLU.
Current recommendation	Do not use this variant as the main Logos architecture.

10. One-Line Summary

The deeper-narrower Logos model reduced parameters and checkpoint size slightly, but produced worse validation quality, slower training, slower generation, and higher energy use; therefore, it is not better than the original baseline or the current best Weight Tying + SwiGLU model.

Key numeric summary

Experiment	Params	Final val loss	PPL	Checkpoint	Latency
Baseline	32.16M	5.7634	318.42	122.71 MB	10.20 ms/token
Deeper-Narrower	29.88M	5.8030	331.29	114.02 MB	13.65 ms/token
Best so far: WT + SwiGLU	19.24M	5.7247	306.36	73.44 MB	15.04 ms/token

Recommended next step

Use Experiment 5 as the current best architecture and test targeted improvements from that branch, rather than continuing with the deeper-narrower branch. A useful next experiment would be a data-quality experiment or a compact inference-focused variant of Weight Tying + SwiGLU.